

5 AI Projects That will Matter in 2026

1. RAG with RBAC, Guardrails and Monitoring

RAG (Retrieval Augmented Generation) is widely used in AI projects worldwide. In our AI consultancy company AtliQ, majority of AI implementations we have worked on for our clients based in USA and UAE are RAG projects.

LLMs have knowledge cut-off date, for example GPT 5.4 is trained on data till Aug 2025. If you want to build AI application that uses GPT 5.4 as a brain and reasons over some private organizational data or public information post Aug 2025, you will have to use RAG.

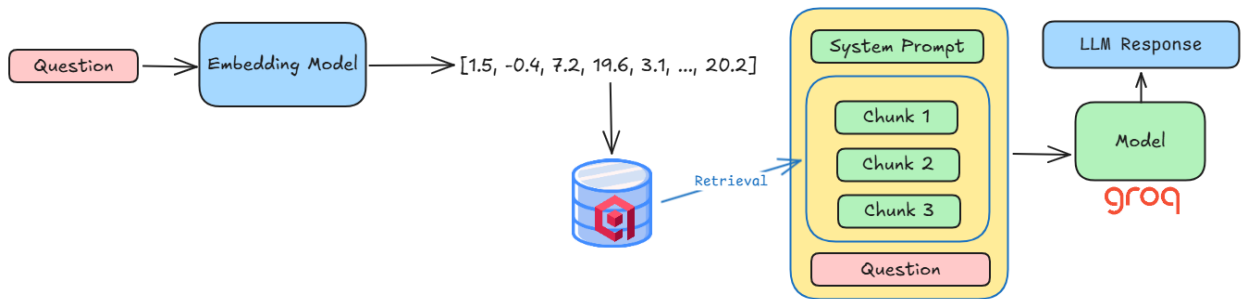
What you will build

- Build internal chatbot for a company that can answer questions using company's private data
- Limit the access of information using Role Based Access Control (RBAC) which means only finance team gets access to financial reports, marketing expenses. Only HR team can access employee data and payroll. Only C-level executives have full access to all company data
- **Setup guardrails:** Protect sensitive / PII data, Out-of-scope question detection
- **Deploy in Azure.** Setup monitoring and evaluation so that whenever a new change is deployed it will run evals to make sure things don't break.
- **Cost monitoring:** Setup tracking for token cost, raise alerts when needed

Tech Stack

- **Core framework:** Langchain and/or Docling along with Python
- **Vector Database:** Any vector store such as Qdrant, Chromadb or Milvus
- **Frontend:** Streamlit, or vibe code react web app using Claude Code
- **LLM:** GPT-OSS or Llama (Use it in groq cloud for free)
- **Cloud:** Azure or AWS or GCP
- **Evaluation and Monitoring:** Ragas, Langsmith

RAG Pipeline



Resources

- Download problem statement, datafiles etc. from here: <https://codebasics.io/challenges/codebasics-gen-ai-data-science-resume-project-challenge/19>
- Vektordb explained: <https://youtu.be/72XgD322wZ8?si=8qplwk5ZpOG-Rms7>
- Groqcloud: <https://console.groq.com/>
- Ragas: <https://docs.ragas.io/en/stable/>
- LangSmith: <https://smith.langchain.com/>
- RAG in Langchain: <https://docs.langchain.com/oss/python/langchain/rag>
- Guardrails in Langchain: <https://docs.langchain.com/oss/python/langchain/guardrails>

2. Voice Agent

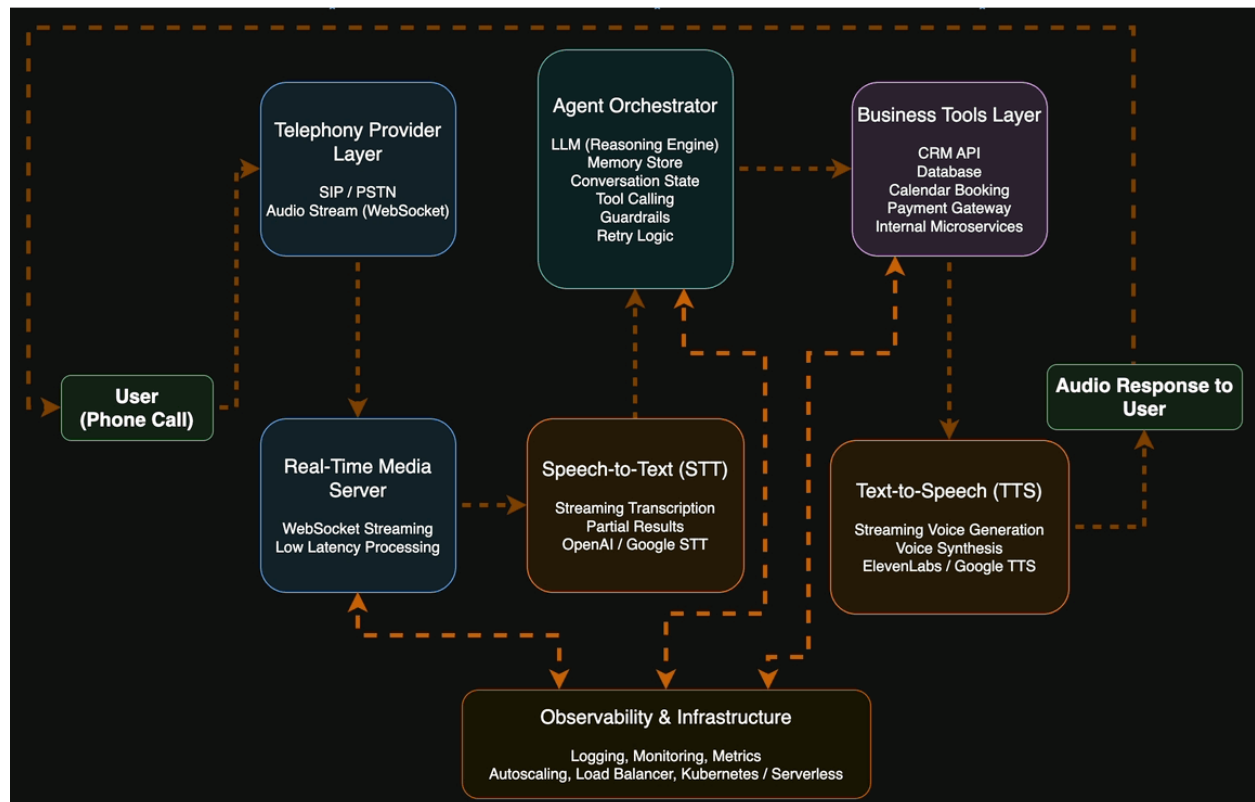
Customer service phone calls can be automated by building voice agents.

What you will build

- Voice agent should be able to listen to a caller, convert speech to text, understand the customer's intent using an LLM, retrieve relevant information from company knowledge bases (using RAG), and respond back in natural speech using text-to-speech.
- Such an agent can answer FAQs, check order status, schedule appointments, or route complex issues to a human agent.

Tech Stack

- **Voice platform (Commercial):** Elevenlabs (their free plan has 10K free credits per month) or Vapi
- **Voice platform (opensource):** Livekit
- **Core framework:** Python with LangChain / LangGraph for orchestration and agent flow
- **Telephony:** Twilio to buy a phone number and manage inbound / outbound calls
- **Vector Database:** Any vector store such as Qdrant, ChromaDB, or Milvus for knowledge retrieval
- **Frontend:** Streamlit dashboard, or vibe-code React web app using Claude Code
- **LLM:** GPT-OSS or Llama for reasoning and response generation, hosted on Groq Cloud for free where applicable
- **STT (Speech to text) providers:** Deepgram, Cartesia
- **TTS (Text to speech) providers:** Elevenlabs, Cartesia, vapi
- **Cloud:** Azure or AWS or GCP





Resources

- Build voice agent with Elevenlabs: <https://youtu.be/fnivYSh0Cqk?si=kJgfXEjnMuyy46pT>
- Vapi: <https://vapi.ai/>
- Elevenlabs: <https://elevenlabs.io/>
- Twilio: <https://www.twilio.com/>

3. Multi-Agent Coding Assistant

Modern AI systems are moving toward **agentic software development**, where AI behaves like a small engineering team instead of a single chatbot. In this project you will build an **AI coding assistant** that can take a natural language request and generate a working software project automatically.

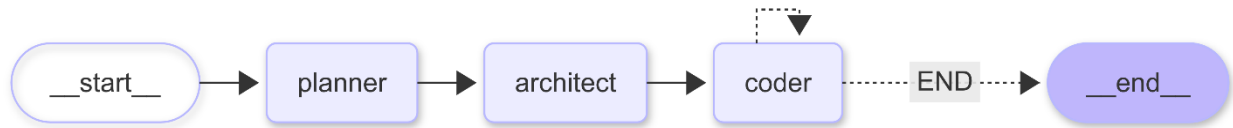
The idea is inspired by the **Coder Buddy project**, an AI-powered coding assistant built with LangGraph that behaves like a multi-agent development team capable of converting a prompt into a working project file-by-file.

For example, a user could type:

“Build a calculator web app with add, subtract, multiply and divide buttons.”

codebasics.io

The system will automatically plan the architecture, create required files, and generate code for the application.



What you will build

- Build an AI coding assistant that converts natural language requests into working code projects
- Implement a Planner Agent that analyzes the user request and creates a project plan
- Implement an Architect Agent that breaks the plan into specific engineering tasks for each file
- Implement a Coder Agent that generates code and writes it directly into project files
- Allow the system to generate multi-file applications such as web apps or APIs
- Provide an interface where users can download the generated project or run it locally

Tech Stack

- **Agent framework:** LangGraph or CrewAI with Python
- **Frontend:** Streamlit, or vibe code react web app using Claude Code
- **LLM:** Kimi K2, Qwen 3 (Use it in groq cloud for free)

Resources

- Guided YouTube Tutorial: https://youtu.be/SP-b_G74Nuk?si=HBvg!Oa9nBbayGdT

4. Multimodal AI assistant

Build a multimodal AI assistant where user can attach image and AI will assist with the queries. Many finance and operations teams often see a charge in a credit card statement, invoice, or expense report and want a quick explanation. In this project you will build an AI assistant where a user uploads a financial document screenshot or PDF and asks a question like “Why was this charge deducted?”



Credit Card Statement

Statement Date: 12/05/2026

Date	Description	Amount	Balance
12/02/2026	Payment	\$300.00	\$5,660.00
12/02/2026	Amazon Web Services EC2 Usage	\$320.45	\$4,512.78

Previous Balance: \$4,215.60

Payments Received: - \$600.00

New Charges: \$947.68

Amazon Web Services - EC2 Usage \$320.45

Netflix Subscription \$15.99

Grocery Store Purchase \$74.32

Taxes & Fees: \$24.50

Total Due: \$4,512.78



Why was this charge deducted?



Send



This charge of \$320.45 was for Amazon Web Services EC2 Usage on your account.

You have been billed for the usage of AWS EC2 virtual servers, which are typically used for cloud computing resources.

This likely reflects your usage of *AWS cloud* services for the past billing period.

The system uses vision + document extraction + retrieval + reasoning to understand the document and explain the charge.

What You'll build

- Upload a screenshot of a **credit card statement, invoice, receipt or expense report**
- Extract fields such as **vendor name, line item, subtotal, tax, total amount and billing date**
- Use a vision-capable LLM to understand the document layout and identify the charge
- Retrieve supporting context such as **prior invoices or billing policies** using a vector database
- Generate a grounded answer explaining the reason for the charge
- Deploy the application in cloud and monitor performance, accuracy and cost

Tech Stack

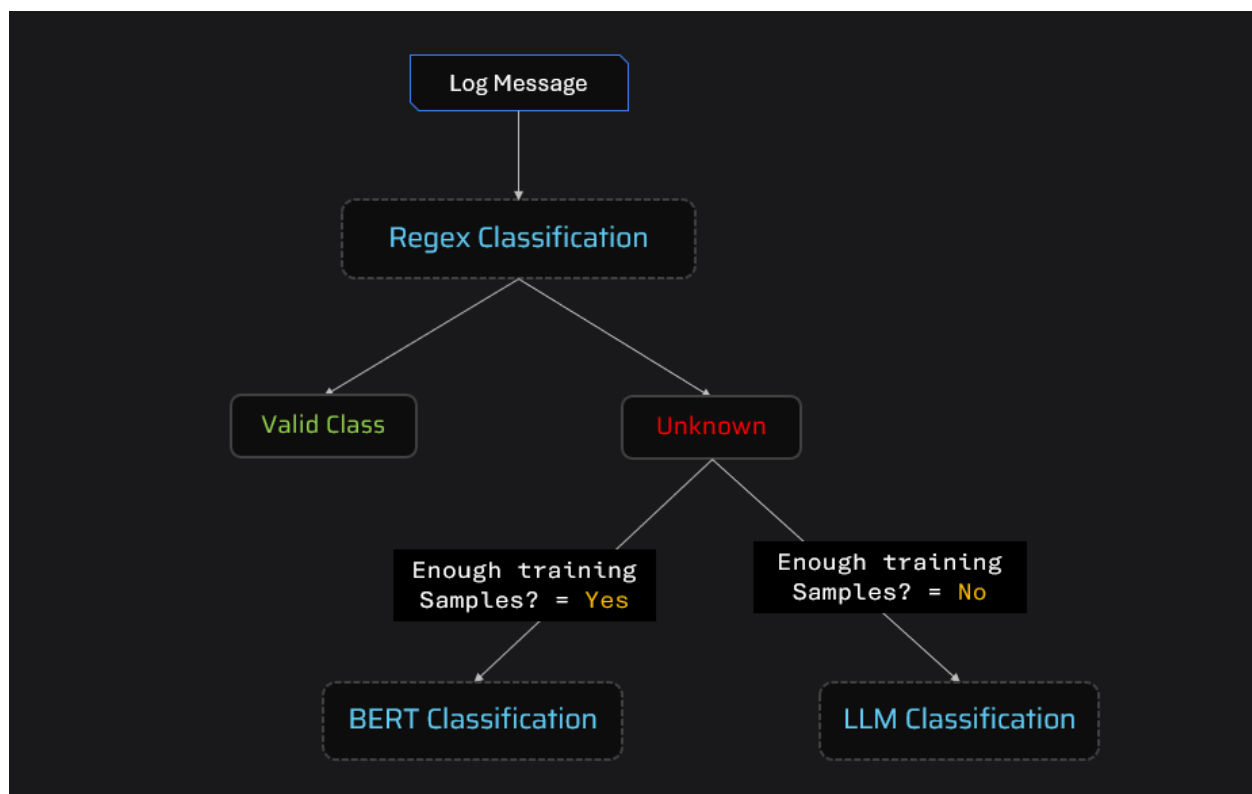
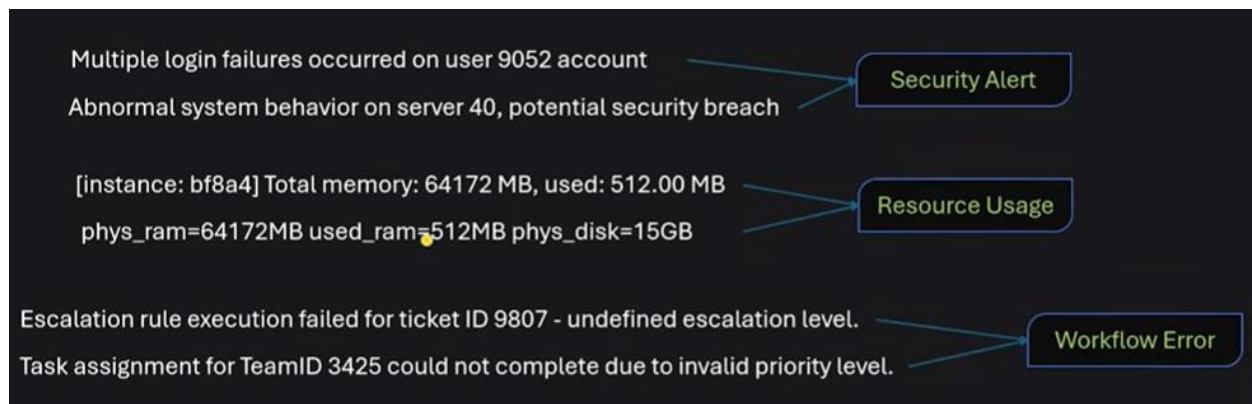
- **Core framework:** Langchain and/or Docling along with Python
- **Vector Database:** Any vector store such as Qdrant, Chromadb or Milvus
- **Frontend:** Streamlit, or vibe code react web app using Claude Code
- **LLM:** GPT-OSS or Llama (Use it in groq cloud for free)
- **Cloud:** Azure or AWS or GCP
- **Evaluation and Monitoring:** Ragas, Langsmith

Resources

- **Docling:** <https://docling-project.github.io/docling/concepts/chunking/>

5. Text Classification using Regex + Statistical ML + LLM

Text classification remains a fundamental need for many AI applications. In this project, you will build a hybrid classification model using regular expression, statistical ml and LLM to classify programming logs.



What You'll build

- Classify programming logs using a hybrid approach
- Text that has fixed patterns will be classified using regular expression
- Text that has variable patterns but enough labeled data will be classified using BERT and LogisticRegression
- Text that has complex patterns with very few labeled data points will be classified using LLM

Tech Stack

- **Core framework:** Python, Sklearn, Pandas, Groq
- **Frontend:** Streamlit
- **Backend:** FastAPI
- **LLM:** Deepseek or Qwen

Resources

- **YouTube tutorial:** https://youtu.be/Ji3_VX80YJg?si=fBcRtROakKSuNp2u
- **Additional Dataset:** <https://www.kaggle.com/search?q=text+classification>